

Influencia del crecimiento y del umbral de carga

Tablas hash dinámicas con hashing universal

José Cisternas

Gabriel Olarte

Bernardo del Aguila

Wara Murillo

Algorítmica II – Universidad Privada Boliviana

11 de septiembre de 2026

Resumen

Este trabajo estudia cómo el factor multiplicativo de crecimiento γ y el umbral de carga τ modifican el costo amortizado, la memoria y las colisiones de una tabla hash dinámica. Se implementó en Python una tabla con encadenamiento separado, redimensionamiento geométrico y una familia universal modular; como contraste, se adaptó una tabla con sondeo lineal. El diseño factorial evaluó cinco factores de crecimiento, cuatro umbrales, tres tamaños de instancia y 30 repeticiones independientes, para 1800 ejecuciones principales. En $N = 100000$, aumentar γ de 1.25 a 4 redujo las migraciones medias de 4.508 a 0.723 por inserción, mientras elevó la memoria estructural final de 157.7 a 276.8 bytes por elemento. La correlación de rangos entre γ y migraciones fue -1.000 ; entre τ y pares que colisionan fue 0.949. El cociente medio entre colisiones observadas y la cota esperada universal fue 0.964. Los resultados respaldan el compromiso previsto: crecer más reduce reconstrucciones, pero reserva más memoria; un umbral alto incrementa la carga y las colisiones. No existe una configuración óptima independiente del objetivo y del entorno.

Palabras clave: hashing universal; análisis amortizado; tablas hash dinámicas; colisiones; reproducibilidad.

1. Introducción

Las tablas hash proporcionan acceso eficiente cuando las claves se distribuyen adecuadamente. En una implementación dinámica, sin embargo, no basta con elegir una función de dispersión: también debe decidirse cuándo reconstruir la tabla y cuánto crecer. Un crecimiento pequeño conserva espacio, pero repite migraciones; uno grande reduce reconstrucciones, aunque deja capacidad ociosa. De manera análoga, un umbral bajo acorta cadenas a costa de reservar más buckets.

El problema es algorítmicamente relevante porque combina dos tipos de garantía que suelen confundirse. El costo amortizado distribuye el trabajo de reconstrucciones ocasionales sobre una secuencia completa, sin suponer una distribución probabilística. El hashing universal, en cambio, ofrece una esperanza respecto de la función seleccionada aleatoriamente [4, 12]. La brecha que abordamos es observar ambas garantías en una misma implementación instrumentada, manteniendo separadas sus unidades de medida.

1.1. Pregunta de investigación

¿Cómo influyen el factor de redimensionamiento γ y el umbral de carga τ en las migraciones amortizadas, el consumo de memoria y el número esperado de colisiones de una tabla hash dinámica con hashing universal?

1.2. Hipótesis o expectativa contrastable

H1. Al aumentar γ disminuirán el número de redimensionamientos y las migraciones por inserción aproximadamente como $1/(\gamma - 1)$, pero aumentarán la reserva y el pico de memoria. Al aumentar τ crecerán la carga y las

colisiones. Las métricas formarán una frontera de compromiso, no un ganador universal.

H0. Una vez controlada la carga realizada α , γ y τ no producirán un efecto directo adicional sobre los pares que colisionan; los residuos se explicarán por trayectoria, redondeo a primos y variabilidad entre funciones hash.

1.3. Contribuciones

- **Teórica:** una derivación que distingue migraciones amortizadas, colisiones esperadas y promedios empíricos.
- **Experimental:** un factorial reproducible de 1800 ejecuciones, métricas separadas, intervalos bootstrap y ocho visualizaciones finales.
- **Integración:** una matriz completa del bloque final y un contraste con sondeo lineal que evita trasladar métricas entre esquemas incompatibles.

2. Trabajos relacionados / estado del arte

Carter y Wegman introducen familias universales para controlar colisiones sin asumir claves aleatorias [4]. Tarjan organiza el análisis amortizado de secuencias [12], mientras Larson relaciona carga, expansión y reorganización en tablas dinámicas [7]. Estos trabajos fundamentan las variables principales, pero no determinan una política única para una implementación moderna.

Otros esquemas muestran por qué las conclusiones dependen de la representación. El hashing perfecto dinámico busca consultas de peor caso [5]; cuckoo hashing usa reconstrucción aleatoria ante ciclos [9]; el análisis de sondeo lineal revela una dependencia pronunciada de la carga [6]. La independencia requerida también cambia según la aplicación [10].

La literatura reciente estudia alta carga, concurrencia y arquitectura: DyCuckoo se orienta a GPU [8], Iceberg hashing optimiza varios criterios simultáneos [1], y los benchmarks vectorizados evidencian que el hardware puede alterar los rankings [2]. Los arreglos redimensionables delimitan, además, la frontera tiempo-espacio del crecimiento geométrico [3, 11].

2.1. Síntesis crítica

La evidencia converge en tres puntos: la carga afecta las colisiones, la reconstrucción distribuye un costo ocasional y ninguna política domina todas las métricas. No obstante, las garantías de encadenamiento, sondeo lineal, cuckoo hashing y estructuras para GPU no son intercambiables. Nuestro estudio no intenta reproducir cada arquitectura; aísla γ y τ en una implementación controlada y usa los demás enfoques como contexto o contraste.

3. Fundamento teórico y algoritmos

3.1. Modelo del problema

La entrada es una secuencia de N claves enteras únicas y un tratamiento (γ, τ) , con $\gamma > 1$ y $0 < \tau < 1$. La tabla tiene m buckets, n elementos y carga $\alpha = n/m$. La salida experimental es un vector de métricas: migraciones, redimensionamientos, comparaciones, pares que colisionan, memoria y tiempo. Las semillas de claves, orden y hash se registran por separado.

La familia usada es

$$h_{a,b}(x) = ((ax + b) \bmod p) \bmod m, \quad p = 2^{61} - 1,$$

con $a \in \{1, \dots, p-1\}$ y $b \in \{0, \dots, p-1\}$. En cada reconstrucción se renuevan (a, b) . Para longitudes de cadena L_j , el conteo de pares es

$$C_{\text{pares}} = \sum_{j=0}^{m-1} \binom{L_j}{2}, \quad \mathbb{E}[C_{\text{pares}}] \leq \frac{n(n-1)}{2m}.$$

La desigualdad es una esperanza sobre la elección de h , no una cota determinista para cada semilla.

3.2. Algoritmo/enfoque principal

La implementación usa encadenamiento separado y capacidad inicial prima $m_0 = 11$. Antes de insertar una clave nueva, verifica si $(n+1)/m > \tau$. Si la condición se cumple, asigna

$$m' = \text{next_prime}(\lceil \gamma m \rceil),$$

selecciona una nueva función universal y migra todas las entradas. Después calcula el bucket, descarta duplicados e inserta la clave. El contador incremental de pares agrega L_j al insertar en una cadena de longitud L_j .

Pseudocódigo de inserción. (1) Si la clave ya existe, terminar. (2) Si la próxima carga supera τ , crear una tabla de capacidad m' , renovar el hash y migrar cada clave. (3) Evaluar $h(x)$. (4) Sumar al contador de pares la longitud actual del bucket. (5) Insertar y actualizar métricas e invariantes.

3.3. Línea base o alternativa de comparación

El contraste usa direccionamiento abierto con sondeo lineal, adaptado del prototipo de la rama gabo. Allí una “colisión” es un probe sobre un slot ocupado y puede aparecer clustering primario. La comparación es pertinente para mostrar el efecto de τ , pero no equipara probes con pares de claves en un bucket.

3.4. Corrección, complejidad y garantías

La conservación se obtiene porque cada reconstrucción recorre todas las claves y las reinserta con la nueva función; la unicidad se preserva al comprobar pertenencia antes de insertar. Tras cada operación se verifica $n = \sum_j L_j$, $\alpha \leq \tau$ después de la eventual expansión y la igualdad entre el contador incremental y $\sum_j \binom{L_j}{2}$.

Si una tabla de tamaño m_j crece a aproximadamente γm_j cuando contiene τm_j entradas, el ciclo siguiente admite cerca de $\tau m_j(\gamma - 1)$ inserciones. La reconstrucción mueve cerca de τm_j entradas; por método agregado, la migración amortizada dominante es

$$\frac{\tau m_j}{\tau m_j(\gamma - 1)} = \frac{1}{\gamma - 1} = O(1).$$

La interpretación contable asigna créditos a cada inserción para financiar la próxima migración. Una potencial basada sólo en ocupación no captura limpiamente el redondeo a primos y la combinación (γ, τ) , por lo que se usa como contraste y no como prueba principal.

Después de crecer, $\alpha_{\text{post}} \approx \tau/\gamma$: los buckets por elemento se aproximan a γ/τ y el pico estructural, mientras coexisten ambas tablas, a $(1 + \gamma)/\tau$. Estas expresiones predicen un costo espacial creciente con γ .

4. Integración del bloque final

Análisis amortizado, probabilidad y esperanza, y algoritmos aleatorizados son centrales: explican, respectivamente, las migraciones sobre secuencias, el conteo esperado de colisiones y la selección aleatoria de la función

hash. La implementación siempre conserva la respuesta correcta; por ello se aproxima al patrón Las Vegas en corrección, aunque el estudio mide una estructura y no clasifica un algoritmo de búsqueda completo.

P, NP, reducciones, aproximación y heurísticas funcionan como contraste conceptual. El problema experimental no es un lenguaje de decisión NP-completo ni aproxima un óptimo NP-difícil. Esta exclusión es técnica: una analogía entre buckets y grafos no constituye una reducción. La cobertura por subtema, incluso para estados N, aparece en el Apéndice A.

Cuadro 1: Papel global de las unidades temáticas.

Unidad temática	Estado	Papel en la investigación
Análisis amortizado	A	Cota de migraciones en secuencias de inserciones.
Probabilidad y esperanza	A	Indicadoras y cota esperada de pares que colisionan.
Algoritmos aleatorizados	A	Selección y renovación de una función universal.
P, NP y verificadores	P	Contraste entre verificación polinomial y la pregunta empírica.
Reducciones y NP-completitud	N	No se demuestra dureza ni se transforman lenguajes.
Aproximación y heurísticas	P	Pareto orienta selección, sin razón de aproximación.

A = aplica sustantivamente; P = aplica parcialmente/complementa; N = no aplica con justificación.

5. Metodología

5.1. Diseño experimental

El factorial cruza $\gamma \in \{1.25, 1.5, 2, 3, 4\}$, $\tau \in \{0.50, 0.70, 0.80, 0.90\}$ y $N \in \{10^3, 10^4, 10^5\}$. Cada celda usa 30 semillas: 1800 unidades experimentales. Dentro de cada bloque, los tratamientos reciben el mismo conjunto y orden de claves; el orden de tratamientos se aleatoriza para repartir deriva térmica y carga externa. Un piloto independiente de cinco semillas se conserva en archivos separados.

Cuadro 2: Plan experimental.

Factor	Valores	Métricas principales	Repeticiones
N	$10^3, 10^4, 10^5$	tiempo, memoria, migraciones	30 por celda
γ	1.25, 1.5, 2, 3, 4	resizes, migraciones, bytes	30 por celda
τ	0.50, 0.70, 0.80, 0.90	carga, pares, comparaciones	30 por celda
Esquema secundario	sondeo lineal	probes por inserción	540 corridas

Las variables dependientes incluyen elementos migrados, evaluaciones hash, comparaciones, eventos de bucket ocupado, pares finales, longitud máxima de cadena, slots estables/pico, bytes estructurales, pico de `tracemalloc`, nanosegundos y percentiles de latencia. Se usa `perf_counter_ns`; el calentamiento se ejecuta fuera de la región medida y el estado del recolector queda en el CSV. El entorno exacto se registra en `environment.txt`.

Las pruebas automatizadas cubren casos normales, claves duplicadas, umbral inminente, capacidad prima, renovación del hash, conservación y equivalencia del contador de pares. El criterio para respaldar H1 es observar tendencias monotónicas agregadas coherentes con las predicciones y un compromiso visible entre migraciones, memoria y colisiones; no se exige que cada semilla individual sea monotónica.

5.2. Reproducibilidad

Desde la raíz, `powershell -File scripts/reproduce.ps1` ejecuta pruebas, experimentos, validación, resúmenes y figuras. `powershell -File scripts/compile_documents.ps1` recompila artículo y bitácoras. El `README.md` explica estructura y comandos; `data/raw/` conserva observaciones, `data/processed/` resúmenes y `figuras/final/` las salidas gráficas. Cada fila registra las semillas, el commit, Python, plataforma y CPU.

6. Resultados

La corrida definitiva contiene **1800** unidades experimentales y 30 repeticiones por celda. Para $N = 100000$, la correlación de rangos entre γ y migraciones por inserción, promediando τ , fue -1.000; entre τ y colisiones por pares por clave, promediando γ , fue 0.949. El cociente medio entre pares observados y la cota universal fue 0.964. Estos valores describen la muestra de semillas y no convierten una esperanza en cota determinista.

La menor migración media apareció en $(\gamma, \tau) = (4, 0.70)$; el menor uso estructural final en $(1.5, 0.90)$; y la menor tasa de pares en $(4, 0.50)$. Que estos puntos no sean necesariamente el mismo tratamiento aporta evidencia empírica del compromiso multiobjetivo.

Cuadro 3: Promedios por γ para $N = 100000$, agregados sobre τ y semillas.

γ	migr./ins.	bytes/elem.	pares/clave	resizes	ns/oper.
1.25	4.508	157.7	0.333	40.2	66080.8
1.5	2.413	166.1	0.289	23.0	43657.2
2	1.491	187.3	0.222	14.0	33174.9
3	0.929	218.5	0.204	9.0	29056.5
4	0.723	276.8	0.189	7.2	27944.1

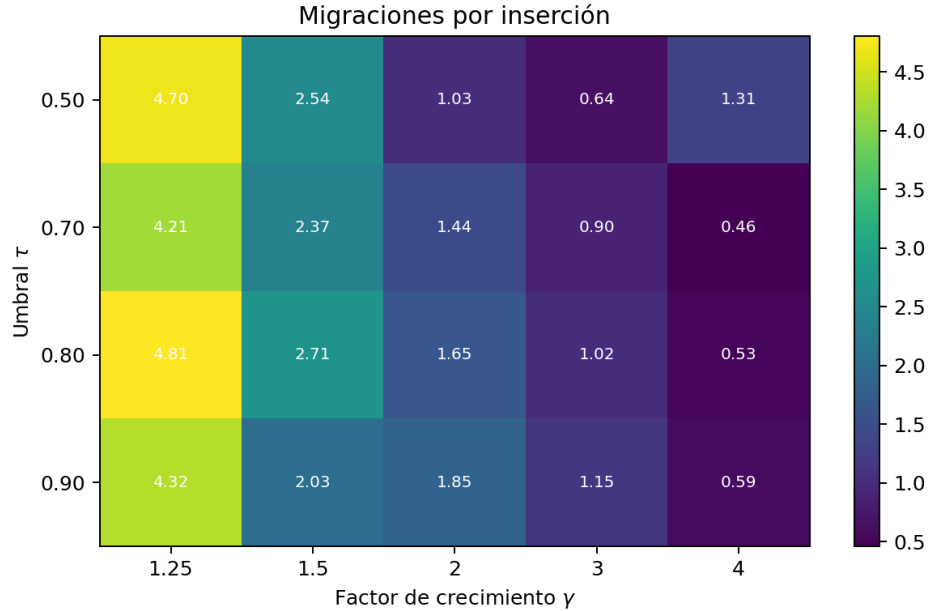


Figura 1: Migraciones por inserción para $N = 100000$. Los valores disminuyen al crecer γ .

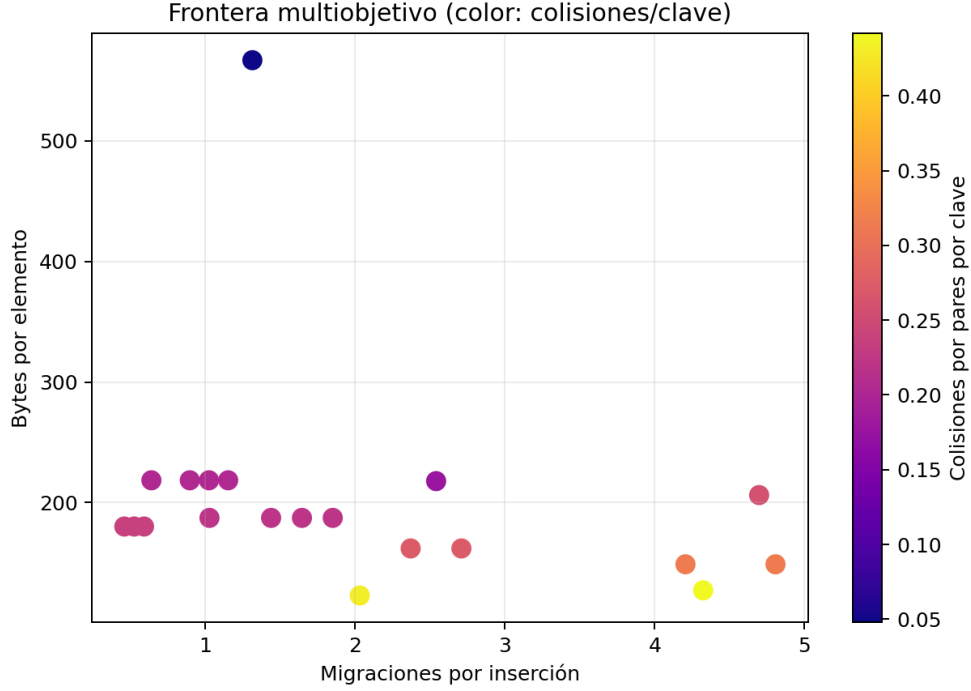


Figura 2: Compromiso entre migraciones, memoria y colisiones. Ningún tratamiento minimiza simultáneamente las tres métricas.

El contraste de sondeo lineal contiene 540 corridas. Para $N = 10000$, promediando factores de crecimiento y 30 semillas, los probes por inserción fueron 1.75, 3.46 y 8.26 para $\tau = 0.50$, 0.75 y 0.90. Se informa como resultado separado porque su noción de colisión difiere de C_{pares} .

7. Discusión

La caída de migraciones con γ respalda la predicción amortizada $1/(\gamma - 1)$. A la vez, el crecimiento de bytes por elemento muestra el precio espacial anticipado por γ/τ . El mejor tratamiento depende, por tanto, de si se prioriza memoria, reconstrucciones o colisiones. La Figura 2 hace visible esta ausencia de dominación total.

La correlación positiva entre τ y pares concuerda con la mayor carga. Sin embargo, τ es un disparador, no la carga final: el redondeo a primos y la trayectoria pueden hacer que dos políticas terminen con igual capacidad o presenten inversiones en un N concreto. El cociente 0.964 frente a la cota universal es coherente en promedio, pero no prueba que cada ejecución deba quedar debajo de ella.

En sondeo lineal, el aumento de probes a alta carga concuerda cualitativamente con el análisis clásico [6]. La diferencia de magnitud respecto del encadenamiento no indica por sí sola superioridad: las métricas, la representación de memoria y el mecanismo de resolución son distintos.

El tiempo por operación sigue en términos generales el volumen de migraciones, aunque incluye el costo de cronometrar cada inserción y mantener `tracemalloc`. Por eso las comparaciones bloqueadas dentro de este entorno son más defendibles que extrapolar nanosegundos absolutos a otra máquina.

8. Limitaciones y amenazas a la validez

La validez de constructo depende de no confundir eventos, pares, comparaciones y probes. La validez interna puede afectarse por GC, calentamiento y procesos del sistema; el bloqueo, la aleatorización y los contadores inde-

pendientes del hardware reducen, pero no eliminan, esas amenazas. La medición con `sys.getsizeof` representa objetos de Python y no una tabla compacta en C.

La validez externa se limita a claves enteras únicas, ejecución monohilo, una familia modular y las distribuciones implementadas. Treinta semillas describen variabilidad, pero no convierten una muestra en demostración. Tampoco se evaluaron borrados, contracción ni adversarios adaptativos. Las referencias identificadas con asistencia de IA deben considerarse verificadas sólo cuando los integrantes hayan consultado sus fuentes originales y puedan defenderlas.

9. Conclusiones y trabajo futuro

La respuesta a la pregunta de investigación es que γ y τ controlan un compromiso medible. En la implementación estudiada, crecer más redujo resizes y migraciones, pero aumentó la reserva de memoria; elevar el umbral aumentó la carga y, agregadamente, los pares que colisionan. H1 queda respaldada para las configuraciones y semillas ensayadas. H0 no puede rechazarse como afirmación causal después de controlar exactamente la carga, porque el factorial intervino políticas completas y no fijó α final de manera independiente.

No puede concluirse que (γ, τ) tenga un óptimo universal ni que los tiempos absolutos se trasladen a otras arquitecturas. Como trabajo futuro se proponen borrados con histéresis, contracción, claves adversas, otras familias hash y una implementación compacta que mida RSS y caché de hardware.

10. Contribuciones y trazabilidad individual

Cuadro 4: Responsabilidades y evidencia disponible al cierre documental.

Integrante	Responsabilidad primaria	Evidencias principales
José Cisternas	Integración, metodología y reproducibilidad	Commit 2efaf13; README.md; protocolo y validación.
Gabriel Olarte	Contraste de sondeo lineal	Commit 69b5543; adaptación en <code>src/linear_probing_hash_table.py</code> .
Bernardo del Aguila	Arquitectura modular y pruebas	Commit 524beb3; <code>src/</code> , <code>tests/</code> y resultados preliminares.
Wara Murillo	Verificación bibliográfica y cobertura	Revisión asignada de anexos y papers núcleo; confirmación individual pendiente.

El equipo coordinó el trabajo mediante ramas y artefactos reproducibles. La tabla no sustituye las bitácoras separadas: cada integrante debe confirmar únicamente las tareas que realizó, corregir cualquier discrepancia y estar preparado para explicar la pregunta, algoritmos, metodología, resultados, matriz y conclusiones.

11. Declaración de uso de IA

Herramienta	Propósito	Prompt representativo	Verificación/modificación
ChatGPT/Codex	Auditar ramas, apoyar código, análisis y LaTeX	“Revisar main y gabo, integrar lo útil y crear entregables conforme a instrucciones e investigación.”	Pruebas automatizadas, validación de CSV, compilación y revisión visual; el equipo debe revisar conceptos y atribuciones.
ChatGPT/Codex	Ajustar a las plantillas oficiales	“Adaptar los entregables a las plantillas PDF oficiales de artículo y bitácora.”	Se reprodujo orden, campos, tablas, apéndices y diagramación; se recompilaron cinco PDF.

La IA no se utilizó como fuente científica. Las referencias sugeridas se contrastaron con metadatos editoriales; la lectura profunda y la responsabilidad final corresponden a los estudiantes. Cada integrante debe comprender lo entregado y poder defender todas las filas de la Matriz Maestra.

Referencias

- [1] Michael A. Bender, Alex Conway, Martín Farach-Colton, William Kuszmaul, and Guido Tagliavini. Iceberg hashing: Optimizing many hash-table criteria at once. *Journal of the ACM*, 70(6):1–51, 2023.
- [2] Maximilian Böther, Lawrence Benson, Ana Klimovic, and Tilmann Rabl. Analyzing vectorized hash tables across CPU architectures. *Proceedings of the VLDB Endowment*, 16(11):2755–2768, 2023.
- [3] Andrej Brodnik, Svante Carlsson, Erik D. Demaine, J. Ian Munro, and Robert Sedgewick. Resizable arrays in optimal time and space. In *Algorithms and Data Structures, WADS 1999*, volume 1663 of *Lecture Notes in Computer Science*, pages 37–48. Springer, 1999.
- [4] J. Lawrence Carter and Mark N. Wegman. Universal classes of hash functions. *Journal of Computer and System Sciences*, 18(2):143–154, 1979.
- [5] Martin Dietzfelbinger, Anna Karlin, Kurt Mehlhorn, Friedhelm Meyer auf der Heide, Hans Rohnert, and Robert E. Tarjan. Dynamic perfect hashing: Upper and lower bounds. *SIAM Journal on Computing*, 23(4):738–761, 1994.
- [6] Philippe Flajolet, Patricio Poblete, and Alfredo Viola. On the analysis of linear probing hashing. *Algorithmica*, 22:490–515, 1998.
- [7] Per-Åke Larson. Dynamic hash tables. *Communications of the ACM*, 31(4):446–457, 1988.
- [8] Yuchen Li, Qiwei Zhu, Zheng Lyu, Zhongdong Huang, and Jianling Sun. DyCuckoo: Dynamic hash tables on GPUs. In *2021 IEEE 37th International Conference on Data Engineering*, pages 744–755. IEEE, 2021.
- [9] Rasmus Pagh and Flemming Friche Rodler. Cuckoo hashing. *Journal of Algorithms*, 51(2):122–144, 2004.
- [10] Mihai Pătraşcu and Mikkel Thorup. The power of simple tabulation hashing. *Journal of the ACM*, 59(3):14:1–14:50, 2012.
- [11] Robert E. Tarjan and Uri Zwick. Optimal resizable arrays. *SIAM Journal on Computing*, 53(5):1354–1380, 2024.
- [12] Robert Endre Tarjan. Amortized computational complexity. *SIAM Journal on Algebraic and Discrete Methods*, 6(2):306–318, 1985.

A. Matriz Maestra de Cobertura Temática

Unidad	Subtema	Estado	Justificación técnica y evidencia
Análisis amortizado	Secuencias de operaciones	A	El costo se evalúa sobre secuencias completas de inserciones; Sec. 2.2 y CSV.
	Método agregado	A	Se suman migraciones de todos los resizes y se normalizan por N ; Sec. 2.2.
	Método contable	P	Los créditos por inserción ofrecen una interpretación de cómo financiar migraciones; no es la prueba principal.
	Método potencial	P	Se discute como alternativa; ocupación sola no captura redondeo primo e interacción (γ, τ) .
	Arreglos dinámicos	A	La reserva geométrica y frontera tiempo-espacio fundamentan el redimensionamiento; Sec. 2.3.
	Contadores y operaciones	A	Resizes ocasionales, elementos movidos y costo por operación están instrumentados en <code>src/</code> .
Probabilidad y esperanza	Probabilidad básica	A	La elección de (a, b) define el espacio aleatorio y los eventos de colisión.
	Esperanza matemática	A	Se contrasta $\mathbb{E}[C_{\text{pares}}] \leq n(n-1)/(2m)$; Sec. 2.1 y Fig. 5.
	Variables indicadoras	A	Cada par aporta una indicadora; su suma produce C_{pares} y permite linealidad de esperanza.
	Amortizado vs. esperado	A	Se distinguen explícitamente garantía de secuencia, esperanza sobre h y media empírica.
Algoritmos aleatorizados	Las Vegas	P	La tabla siempre conserva corrección; la función aleatoria afecta costo, no respuesta.
	Monte Carlo	N	No se acepta probabilidad de respuesta incorrecta: todas las inserciones se verifican.
	Probabilidad de error	N	No existe salida aproximada o errónea que deba amplificarse mediante repetición.
	Garantías de ejecución	P	Se comparan corrección determinista, costo amortizado y colisiones esperadas.
	Selección aleatoria	P	Se eligen aleatoriamente (a, b) y el orden de tratamientos, no pivotes algorítmicos.
	Hashing universal	A	Es el mecanismo aleatorizado central; <code>src/universal_hash.py</code> .
P, NP y verificados	Problemas de decisión	N	La pregunta estima rendimiento multiobjetivo, no decide un lenguaje sí/no.
	Certificados	N	Las observaciones no requieren testigo para una instancia “sí”.
	Verificadores	N	No se formula certificado; la corrección se comprueba mediante invariantes polinomiales.
	Clase P	P	Inserción y validaciones tienen algoritmos polinomiales; no se estudia completitud.
	Clase NP	P	Se explica que pertenencia exige problema de decisión y verificador; esa forma no es central aquí.

Unidad	Subtema	Estado	Justificación técnica y evidencia
	Decisión vs. optimización	P	Elegir una política Pareto es optimización multiobjetivo, distinta de una decisión formal.
Reducciones y NP-completitud	Reducciones polinomiales	N	No se compara dificultad de lenguajes mediante transformaciones de instancias.
	Dirección de una reducción	N	No se afirma dureza; invertir una reducción no demostraría la implicación buscada.
	Prueba de NP-completitud	N	Faltan deliberadamente lenguaje, pertenencia a NP y reducción desde un problema base.
	SAT / 3-SAT	N	No modela la selección de parámetros ni la operación de la tabla.
	Clique	N	Las colisiones forman grupos por bucket, pero no equivalen a clique de una instancia arbitraria.
	Conjunto independiente	N	Evitar colisiones no se formula como selección de vértices sin adyacencias.
	Cobertura de vértices	N	No existe grafo de entrada ni objetivo de cubrir aristas.
	Ciclo hamiltoniano	N	La secuencia de buckets no exige visitar vértices una vez.
	Errores frecuentes	P	Se reconoce que una analogía estructural no constituye reducción ni equivalencia.
	Consecuencias prácticas	P	NP-completitud no implica que problemas polinomiales como esta simulación sean intratables.
Aproximación	Algoritmos de aproximación	N	El factorial mide políticas; no aproxima una solución óptima NP-difícil.
	Razón de aproximación	N	La frontera de Pareto no ofrece razón demostrable respecto de un óptimo único.
	Análisis de garantías	P	Se analizan cotas amortizadas/esperadas, que no deben confundirse con garantía de aproximación.
	Cobertura de vértices	N	No se implementa el algoritmo 2-aproximado ni existe un grafo de entrada.
	Cobertura de conjuntos	N	Los buckets no constituyen una familia de conjuntos cuyo costo de cobertura se optimice.
	Mochila aproximada	N	No hay capacidades, beneficios ni esquema de aproximación para mochila.
Heurísticas y estrategias complementarias	Heurística vs. aproximación	P	La selección visual de un punto Pareto es una guía heurística, no una aproximación con razón probada.
	Búsqueda local	N	No se explora un vecindario de configuraciones para mejorar iterativamente una solución.
	Comparación con solución exacta	N	No existe un óptimo escalar exacto: el objetivo medido es multiobjetivo.
	Calidad de solución	P	Se compara cada política mediante migraciones, memoria y colisiones, sin condensarlas en un puntaje.
	Costo computacional	A	Tiempo, comparaciones, evaluaciones hash y migraciones se registran explícitamente.

Unidad	Subtema	Estado	Justificación técnica y evidencia
	Criterios de selección	P	La frontera de Pareto permite elegir según la prioridad de memoria, colisiones o reconstrucciones.
	Limitaciones	A	La Sec. 8 delimita lenguaje, arquitectura, semillas, instrumentación y generalización.

B. Matriz bibliográfica

Fuente	Problema/objetivo	Método	Resultado relevante	Uso y limitación
Carter–Wegman (1979)	Evitar supuestos benignos sobre claves	Familias universales y análisis probabilístico	Control de colisiones por pares	Fundamenta C_{pares} ; no garantiza probes de sondeo lineal.
Tarjan (1985)	Costear secuencias con eventos caros	Análisis amortizado	Costo medio acotado sobre secuencias	Marco para migraciones; no prescribe (γ, τ) .
Larson (1988)	Hashing dinámico en memoria	Diseño, análisis y experimentación	La expansión controla carga y reorganización	Paper núcleo más cercano; plataforma antigua.
Dietzfelbinger et al. (1994)	Hashing perfecto dinámico	Estructura aleatorizada multinivel	Consultas de peor caso y actualización esperada	Contraste teórico; estructura distinta.
Flajolet et al. (1998)	Costo del sondeo lineal	Combinatoria analítica	La carga cambia el régimen de probes	Respalda experimento secundario; no encadenamiento.
Brodnik et al. (1999)	Arreglos redimensionables eficientes	Diseño y análisis tiempo–espacio	Bajo espacio extra con acceso eficiente	Contexto de reserva; no tabla hash.
Pagh–Rodler (2004)	Búsqueda constante con dos posiciones	Cuckoo hashing	Rehash aleatorio ante ciclos	Relaciona aleatoriedad y reconstrucción; otro esquema.
Pătraşcu–Thorup (2012)	Independencia práctica del hash	Análisis de tabulación simple	Garantías dependen de la aplicación	Evita generalizar universalidad a todo probing.
Richter et al. (2015)	Comparar hashing multidimensionalmente	Benchmark experimental	No hay ganador universal	Motiva factorial y múltiples métricas.
Li et al. (2021)	Hash dinámico en GPU	Diseño concurrente y evaluación	Crecimiento depende de arquitectura	Fuente reciente; no extrapolable a Python.
Bender et al. (2022)	Sondeo lineal a alta carga	Análisis con tombs-tones	Reconstrucción y borrado cambian clustering	Contexto para probing; workload diferente.
Bender et al. (2023)	Optimizar varios criterios	Diseño y análisis de Iceberg hashing	Trade-offs simultáneos	Motiva Pareto; estructura más compleja.
Böther et al. (2023)	Hash vectorizado entre CPU	Benchmark multi-arquitectura	Hardware altera rankings	Amenaza externa; no prueba nuestra política.
Tarjan–Zwick (2024)	Arreglos redimensionables óptimos	Cotas y construcción	Frontera tiempo–espacio refinada	Paper núcleo reciente; no define política hash.

C. Registro de búsqueda bibliográfica

Portal	Fecha	Consulta	Filtros	Resultado/estado
ACM DL	08-09-2026	“universal classes hash functions”, “dynamic hash tables”	Fuente primaria, ve-nue revisado	Carter–Wegman y Lar-son identificados; cada integrante debe confir-mar lectura.
SIAM	08-09-2026	“amortized computatio-nal complexity”, “optimal resizable arrays”	DOI/editor	Tarjan 1985 y Tarjan–Zwick 2024 identifica-dos.
IEEE Xplore	08-09-2026	“dynamic hash tables GPU”, “linear probing tombstones”	2021–2026	Li et al. y Bender et al. seleccionados como contexto reciente.
PVLDB	08-09-2026	“analysis hashing methods”, “vectorized hash tables CPU”	Papers experimenta-les	Richter et al. y Böther et al. para metodología y amenazas.
SpringerLink	08-09-2026	“linear probing analysis”, “re-sizable arrays time space”	DOI estable	Flajolet et al. y Brodnik et al. incorporados.

Nota de integridad. Este registro fue estructurado con asistencia de IA a partir de la investigación suministrada. Antes de la entrega, los estudiantes deben abrir los DOI, consultar los trabajos y registrar en sus bitácoras qué secciones verificaron. Una referencia sugerida por IA no cuenta por sí sola como fuente consultada.

D. Tres papers núcleo

1. **Carter y Wegman (1979).** Problema: evitar supuestos sobre la distribución de claves. Método: familias universales y análisis probabilístico. Resultado: control esperado de colisiones por pares. Limitación: no implica cotas deterministas ni fórmulas de probing. Relación: fundamenta la variable C_{pares} .
2. **Larson (1988).** Problema: expansión de tablas hash dinámicas. Método: diseño y evaluación de reorganiza-ción. Resultado: la política de crecimiento y la carga condicionan costo y espacio. Limitación: plataforma y estructuras diferentes. Relación: antecedente directo del factorial (γ, τ) .
3. **Tarjan y Zwick (2024).** Problema: arreglos redimensionables con buen compromiso tiempo–espacio. Mé-todo: análisis y construcción algorítmica. Resultado: cotas refinadas para reserva dinámica. Limitación: no modela colisiones hash. Relación: contextualiza el costo espacial del crecimiento.

E. Control de bitácoras individuales

Integrante	Entregada	Archivo
José Cisternas	Sí	Jose_Cisternas_bitacora.pdf/.tex
Gabriel Olarte	Sí	Gabriel_Olarte_bitacora.pdf/.tex
Bernardo del Aguila	Sí	Bernardo_del_Aguila_bitacora.pdf/.tex
Wara Murillo	Sí	Wara_Murillo_bitacora.pdf/.tex

F. Checklist de reproducibilidad

- ☐ Código Python incluido.
- ☐ Datos y generadores de instancias incluidos.
- ☐ Semillas aleatorias registradas.
- ☐ Versiones de dependencias indicadas.
- ☐ Instrucciones de ejecución en README.md.
- ☐ Scripts para reproducir tablas y figuras.